

GraphBench Challenge

Réfutation automatique de conjectures en théorie des graphes

GOUASMI Yassine
MOKHTARY Moumen Aya

Université Paris Nanterre
Master 1 MIAGE
Optimisation Combinatoire
Enseignant : François Delbot

09/05/2026

Dépôt GitHub :

<https://github.com/YassineGOUASMI/graphbench-project>

Résultat final : 20/20 contre-exemples trouvés

Score final approximatif : 19.14

Table des matières

1	Introduction	3
2	Architecture du projet	3
2.1	Structure générale	3
2.2	Rôle des principaux fichiers	3
3	Génération et mutations des graphes	4
3.1	Génération initiale	4
3.2	Mutations locales	4
4	Calcul des invariants	4
4.1	Invariants utilisés	4
5	Recherche évolutionnaire	5
5.1	Principe général	5
5.2	Critère de réussite	5
6	Recherche baseline	5
6.1	Extrait du moteur baseline	5
7	Recherche heuristique inspirée de FunSearch	5
7.1	Principe	5
7.2	Extrait du scoring	6
7.3	Observation	6
8	Discussion scientifique	6

1 Introduction

Le challenge GraphBench consiste à rechercher automatiquement des contre-exemples à des conjectures en théorie des graphes.

L'idée générale du projet est de générer des graphes, de calculer plusieurs invariants, puis de vérifier si certaines conjectures du benchmark sont violées.

Pour cela, nous avons développé un programme capable de :

- générer des graphes aléatoires,
- modifier ces graphes avec différentes mutations,
- calculer leurs invariants,
- évaluer la violation d'une conjecture,
- conserver les meilleurs graphes trouvés.

Le système obtenu permet finalement de trouver des contre-exemples pour les 20 premières conjectures testées du benchmark.

2 Architecture du projet

Le projet est organisé en plusieurs modules spécialisés.

2.1 Structure générale

```
graphbench-project/  
|-- benchmark/  
| |-- benchmark.xlsx  
|-- experiments/  
| |-- run_basic_search.py  
| |-- run_baseline_search.py  
| |-- run_targeted_search.py  
| |-- run_final.py  
|-- results/  
| |-- basic_search/  
| |-- baseline_search/  
| |-- targeted_search/  
| |-- final/  
|-- src/  
| |-- conjecture.py  
| |-- generator.py  
| |-- invariants.py  
| |-- mutations.py  
| |-- search.py  
| |-- search_baseline.py  
| |-- scoring.py  
|-- requirements.txt  
|-- README.md
```

2.2 Rôle des principaux fichiers

- `generator.py` : génération initiale des graphes,
- `mutations.py` : opérateurs de mutation,
- `invariants.py` : calcul des invariants,
- `search.py` : moteur de recherche heuristique,
- `search_baseline.py` : baseline classique,
- `scoring.py` : heuristique inspirée de FunSearch,
- `run_baseline_search.py` : expérience principale,

- `run_targeted_search.py` : recherche ciblée,
- `run_final.py` : fusion des meilleurs résultats.

3 Génération et mutations des graphes

3.1 Génération initiale

Le système génère des graphes aléatoires connexes servant de population initiale.

Les tailles de graphes sont variées afin de découvrir des régions différentes de l'espace de recherche.

3.2 Mutations locales

opérateurs de mutation implémentés :

- ajout d'arête,
- suppression d'arête,
- ajout de sommet,
- suppression de sommet,
- ajout de feuille,
- subdivision d'arête,
- ajout de chemin,
- ajout de clique,
- ajout de faux jumeau,
- densification locale.

Ces mutations permettent d'explorer progressivement des graphes de structures variées.

4 Calcul des invariants

Le moteur calcule de nombreux invariants de graphes à l'aide de NetworkX.

4.1 Invariants utilisés

- nombre de sommets,
- nombre d'arêtes,
- degré minimum,
- degré maximum,
- degré moyen,
- diamètre,
- rayon,
- nombre de triangles,
- clique maximum,
- nombre d'indépendance,
- couverture par sommets,
- couplage maximum,
- domination,
- domination indépendante,
- domination totale,
- connectivité par sommets,
- connectivité par arêtes.

5 Recherche évolutionnaire

5.1 Principe général

Le système maintient une population de graphes candidats.

À chaque itération :

1. un graphe parent est sélectionné,
2. une mutation est appliquée,
3. les invariants sont recalculés,
4. la violation de la conjecture est évaluée.

Les meilleurs graphes sont conservés dans la population.

5.2 Critère de réussite

Une conjecture est réfutée lorsque :

$$\text{violation}(G) > 0$$

Le graphe correspondant devient alors un contre-exemple valide.

6 Recherche baseline

La baseline utilise directement la violation réelle comme score principal.

6.1 Extrait du moteur baseline

```
score = conjecture.violation(invariants)

if score > best_score:
    best_score = score
```

Cette approche s'est révélée la plus robuste expérimentalement.

6.2 Capture d'exécution

```
--- Restart 1/3 ---
Score initial : -0.75
Nouveau meilleur score : -0.5
Nouveau meilleur score : -0.25
Nouveau meilleur score : 0.0
Nouveau meilleur score : 0.25

===== MEILLEUR RÉSULTAT BASELINE =====
Trouvé : True
Score : 0.25
Temps : 2.04 sec
Coût : 2.04

=====
Conjecture 2117
If G is a connected graph,  $x = \text{matching\_number}$  and  $y = \text{independent\_}$ 
 $7 * x(G)^2 + 1/3 * x(G)^3$ 
=====

--- Restart 1/3 ---
Score initial : -3.2275714285714194
Nouveau meilleur score : -2.2275714285714194
Nouveau meilleur score : -1.2275714285714194
Nouveau meilleur score : -0.5132857142857077
Nouveau meilleur score : 0.4867142857142923

===== MEILLEUR RÉSULTAT BASELINE =====
Trouvé : True
Score : 0.4867142857142923
Temps : 0.54 sec
Coût : 0.54
```

FIGURE 1 – Exécution de la recherche baseline

7 Recherche heuristique inspirée de FunSearch

Nous avons également testé une approche hybride utilisant un score heuristique supplémentaire.

7.1 Principe

- Le score mélange :
- la violation réelle,
 - des bonus structurels,
 - des pénalités de densité,
 - des critères liés aux invariants.

7.2 Extrait du scoring

```
bonus += 0.01 * triangles
bonus += 0.05 * clique
bonus += 0.02 * domination

return 100.0 * violation + bonus - penalty
```

7.3 Observation

Cette approche a parfois accéléré certaines recherches, mais elle s'est révélée globalement moins stable que la baseline classique.

8 Recherche ciblée

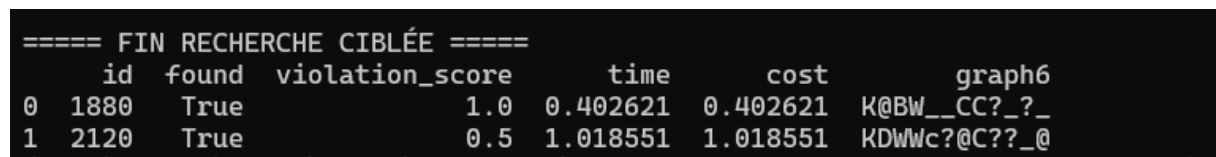
Certaines conjectures restaient difficiles après la baseline.

Nous avons donc implémenté une recherche ciblée dédiée à ces conjectures spécifiques.

8.1 Conjectures ciblées

- 1880
- 2120

8.2 Capture d'exécution



```
===== FIN RECHERCHE CIBLÉE =====
  id  found  violation_score    time    cost    graph6
0  1880   True           1.0  0.402621  0.402621  K@BW__CC?_?_
1  2120   True           0.5  1.018551  1.018551  KDWWc?@C??_@
```

FIGURE 2 – Exécution de la recherche ciblée

Cette stratégie a permis de compléter les derniers contre-exemples manquants.

9 Résultats expérimentaux

9.1 Résultat final

Les expériences ont été réalisées sur les 20 premières conjectures du benchmark, afin de conserver un temps d'exécution raisonnable. Le système a finalement obtenu :

20 / 20 contre-exemples trouvés

9.2 Résumé

Mesure	Valeur
Conjectures testées	20
Contre-exemples trouvés	20
Taux de réussite	100%
Coût approximatif	19.14
Temps moyen	0.96 sec

TABLE 1 – Résumé final des performances

9.3 Capture du résultat final

```
1 2120 True 0.5 1.018551 1.018551 KDWWc?@C??_@
(venv) PS C:\Users\Ce Pc\Documents\graphbench-project> python experiments/run_final.py

===== FINAL RESULTS =====
Conjectures tested : 20
Counterexamples found : 20
Success rate : 100.0 %
Approximate total cost : 19.14
Average time : 0.96 sec

Saved:
results/final/final_results.csv
results/final/final_summary.txt
(venv) PS C:\Users\Ce Pc\Documents\graphbench-project> |
```

FIGURE 3 – Résultat final obtenu

10 Limites

- dépendance à l'aléatoire,
- coût croissant pour les conjectures difficiles,
- absence d'apprentissage automatique réel,

11 Discussion scientifique

Les conjectures les plus faciles à réfuter ont principalement concerné les invariants locaux ou combinatoires tels que le nombre de triangles, le degré maximum, la clique maximum ou le couplage. Ces invariants réagissent fortement aux mutations locales comme l'ajout de clique ou la densification locale.

Les conjectures les plus difficiles ont concerné des invariants plus globaux comme la domination totale, la domination indépendante et la couverture par sommets. Ces valeurs évoluent moins directement lorsqu'on ajoute ou supprime une seule arête.

Les mutations les plus efficaces ont été l'ajout de clique, la densification locale, l'ajout de faux jumeaux et l'ajout de chemins. Elles permettent de modifier rapidement plusieurs invariants tout en conservant souvent la connexité du graphe.

L'approche inspirée de FunSearch, basée sur une fonction de score hybride, n'a pas surpassé la baseline. Elle a parfois guidé la recherche, mais s'est révélée moins stable que l'optimisation directe de la violation. Ce résultat négatif reste intéressant, car il montre que les bonus heuristiques doivent être spécialisés selon les conjectures.

L'utilisation de l'IA générative a surtout été utile pour proposer des structures de code, organiser les expériences, générer des variantes de score et accélérer le développement. Cependant, les meilleures performances ont été obtenues après validation expérimentale et ajustements manuels.

12 Conclusion

Ce projet a permis de construire un moteur complet de recherche automatique de contre-exemples en théorie des graphes.

Les principaux apports sont :

- une architecture modulaire,
- un moteur évolutionnaire robuste,
- plusieurs stratégies de recherche,
- une comparaison entre baseline et heuristiques inspirées de FunSearch.

Les résultats obtenus montrent qu'une approche évolutionnaire relativement simple peut déjà produire des performances très efficaces sur un benchmark de conjectures.

De futures améliorations pourraient inclure :

- des heuristiques adaptatives,
- du reinforcement learning,
- des mutations guidées,
- des modèles génératifs de graphes.